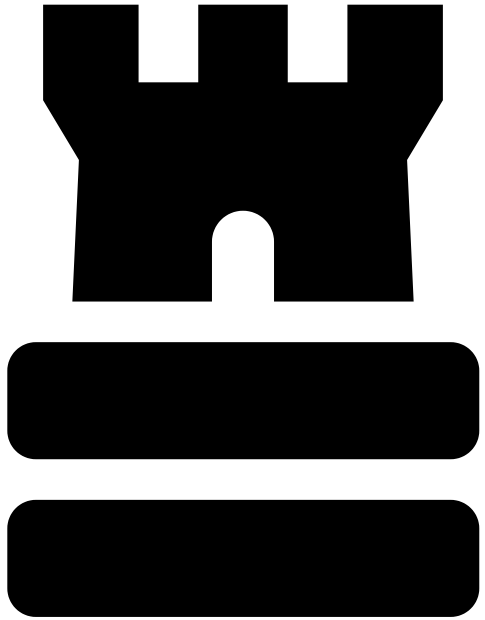
Maskiner som logger inn

Maskinidentitet, delegering og tillit med åpen kildekode

Containerkonferansen | Trondheim, 14. oktober 2026 | Jan Ivar Beddari og Linus Johansen




Kilde: https://tv.nrk.no/program/KOID32005624

Hvilken maskin er dette, og hvem kjører hva på den?

Hvorfor står vi her

- Jan Ivar: UH-sky / NREC, ti år i Safespring med hendelsesdrevne kontrollplan. I 2026: Dataverket, et åpen kildekode-kontrollplan for suveren skyinfrastruktur
- Linus: salmon, en identitetsleverandør for mennesker og arbeidslaster, skrevet fordi de som finnes er bygget for mennesker
- Vi traff samme vegg fra hver vår side: en maskin som skal jobbe på vegne av et menneske, utenfor clusteret den bor i

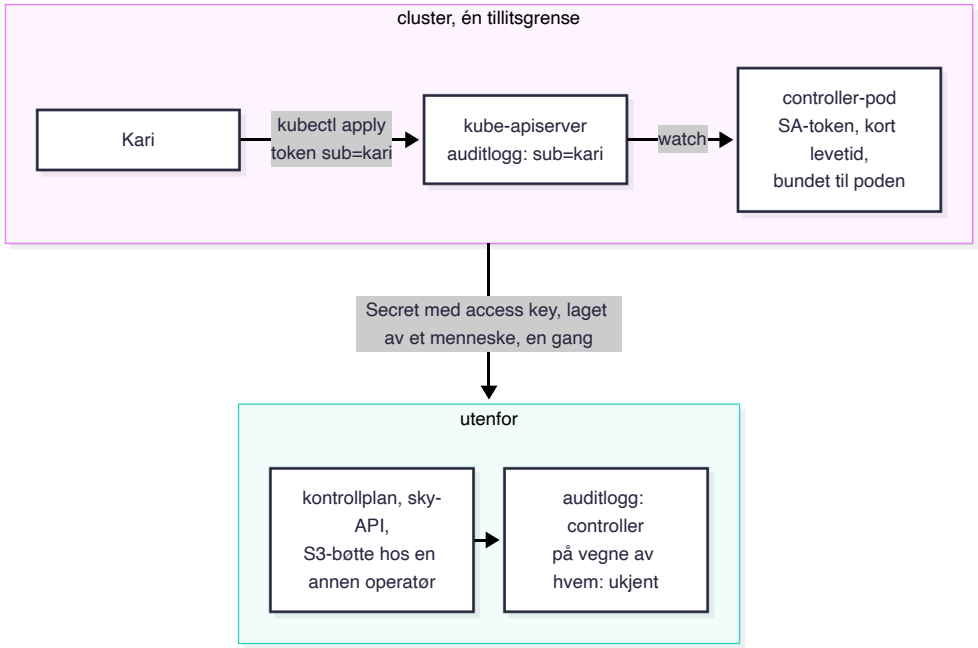
I dag: hvilken prosess på hvilken maskin er dette?

```
{
  "iss": "https://kubernetes.default.svc.cluster.local",
  "sub": "system:serviceaccount:plattform:controller",
  "aud": ["https://kubernetes.default.svc.cluster.local"],
  "exp": 1791979500,
  "kubernetes.io": {
    "namespace": "plattform",
    "serviceaccount": { "name": "controller", "uid": "…" },
    "pod": { "name": "controller-6c9d4-x7k2p", "uid": "…" },
    "node": { "name": "node-07", "uid": "…" }
  }
}
```

Prosess og maskin står i tokenet, men med standardoppsett bare innenfor ett cluster: `iss` er den samme overalt.

Ingen claim sier hvilket menneske som har ansvaret.

I dag: clusteret er grensen



Enten Kari gjør det nå eller en jobb gjør det hver natt i et år: tilgangen ut av clusteret er vanligvis en statisk nøkkel som ble gitt og lagret. De store skyleverandørene har løst dette for sine egne tjenester med workload identity federation. Åpne løsninger tar i beste fall mennesket med så lenge det er logget inn.

Det haster å finne bedre løsninger

- Operatorer, GitOps-controllere og CI-runnere har gjort jobben for oss i ti år. Vi har godtatt at loggen sier «controller»
- En agent velger selv hva den gjør, med nøkler noen ga den én gang
- Loggen sier *hva* som ble gjort og *hvilken maskin* som gjorde det. Den sier sjelden **hvilket menneske som sto bak**

Når maskinen bare gjør det koden sier, kan vi finne mennesket i etterkant, i koden og i ressursen. Når maskinen velger selv, må mennesket finnes i tokenet og logges når det skjer.

Ett problem, to halvdeler

Hvilken prosess på hvilken maskin er dette?

Server, VM eller pod: hvilken organisasjon hører den til, og hvordan beviser den det uten en nøkkel? Det har vi svar på. Jan Ivar viser hvordan Dataverket sitt kontrollplan er bygget.

Men HVEM kjører hva på den??!

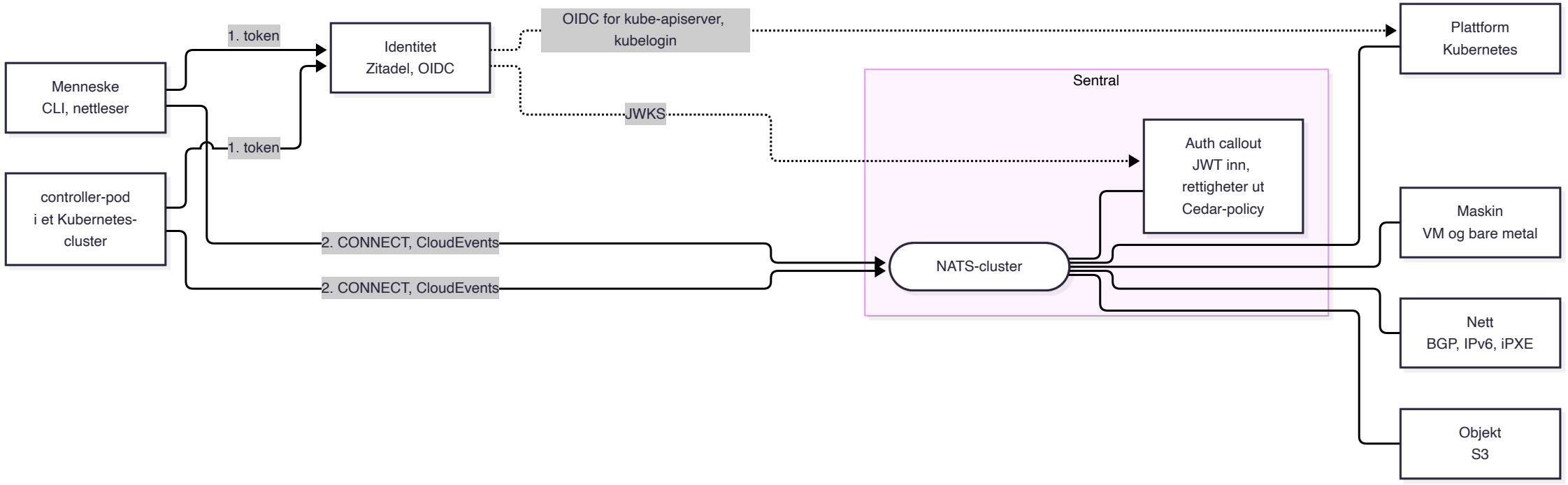
Mennesket med ansvar for prosessen må følge med i tokenet, ut av clusteret og inn i loggen. Dette har ingen gode svar på i dag. Linus har begynt på relevant kode!

Til slutt: Vår ønskeliste til alle, så hver av oss slipper å bygge dette alene.

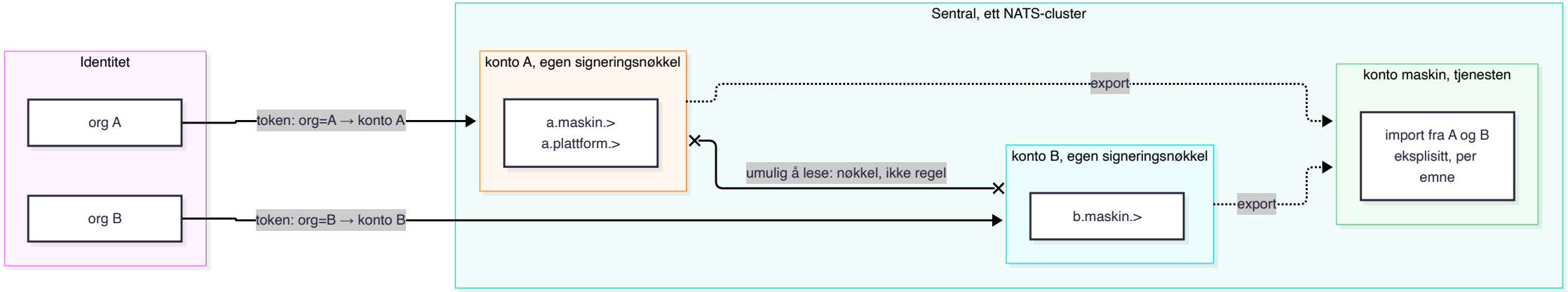
Kontrollplanet

Hvilken prosess på hvilken maskin er dette?

Dataverket på ett bilde

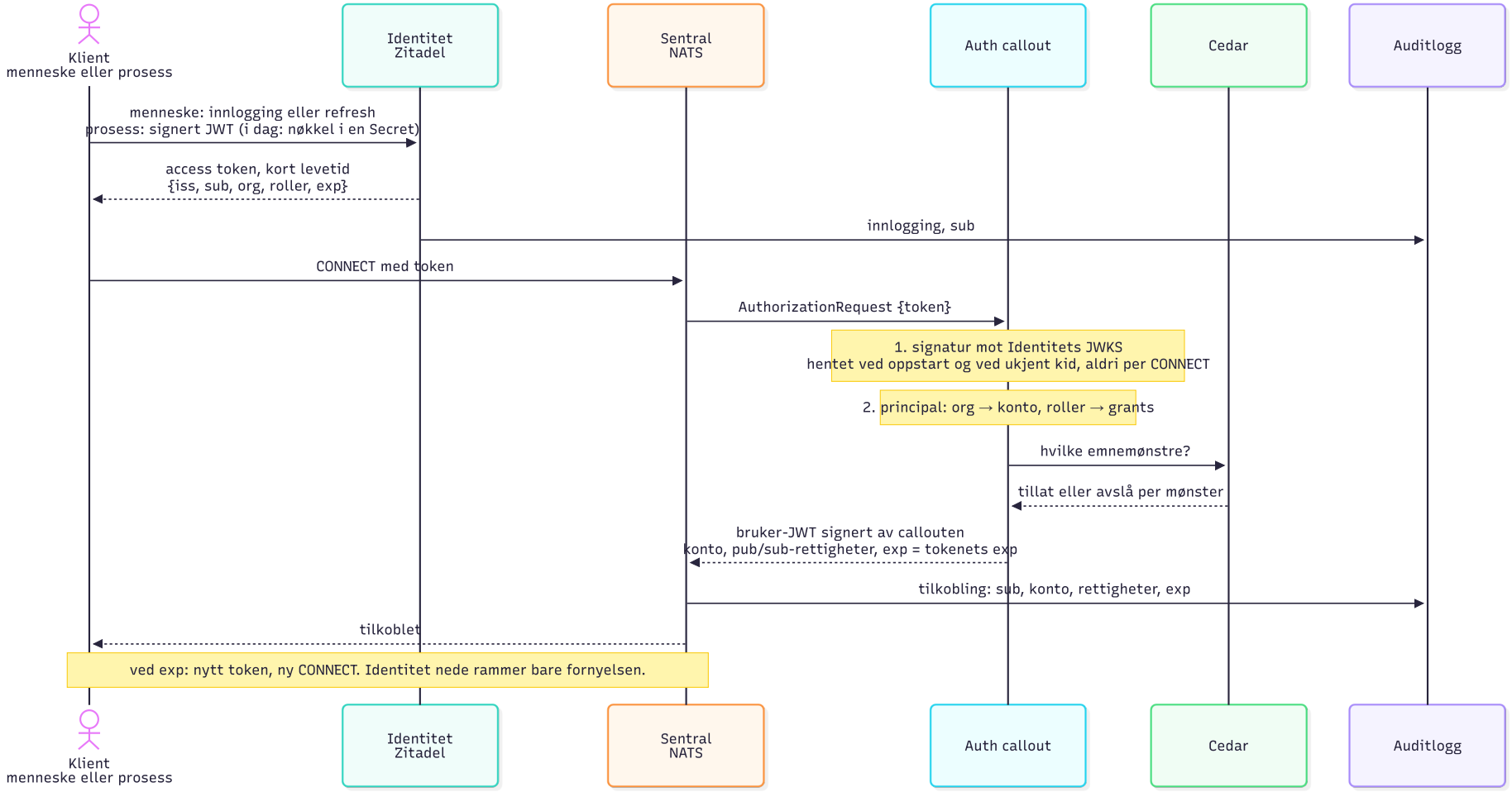


Én organisasjon er ett sikkerhetsdomene



Kubernetes: namespace er en policy-grense. NATS: konto er en nøkkelgrense.

Innslipp: tilkobling til kontrollplanet



En melding i Sentral

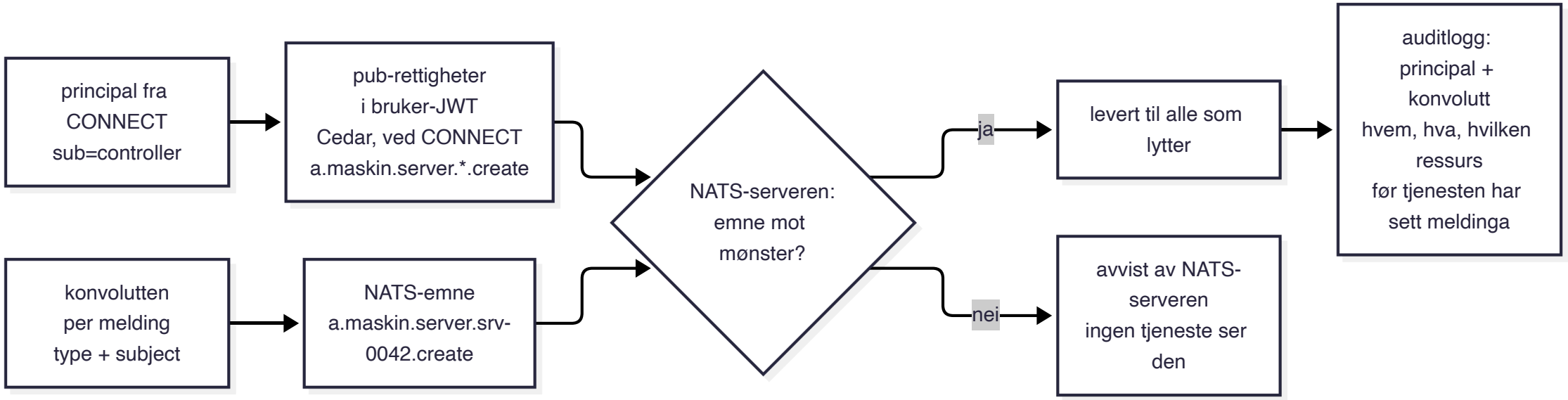
```
{
  "specversion": "1.0",
  "type": "maskin.server.create",
  "source": "dvk:plattform:r1:a:controller/maskin",
  "subject": "dvk:maskin:r1:a:server/srv-0042",
  "id": "…",
  "time": "2026-10-14T11:07:12Z",
  "data": { "cpu": 64, "minne": "512G", "nett": "…" }
}
```

Kommandoer, hendelser, spørringer og svar: samme konvolutt. CloudEvents 1.0, CNCF.

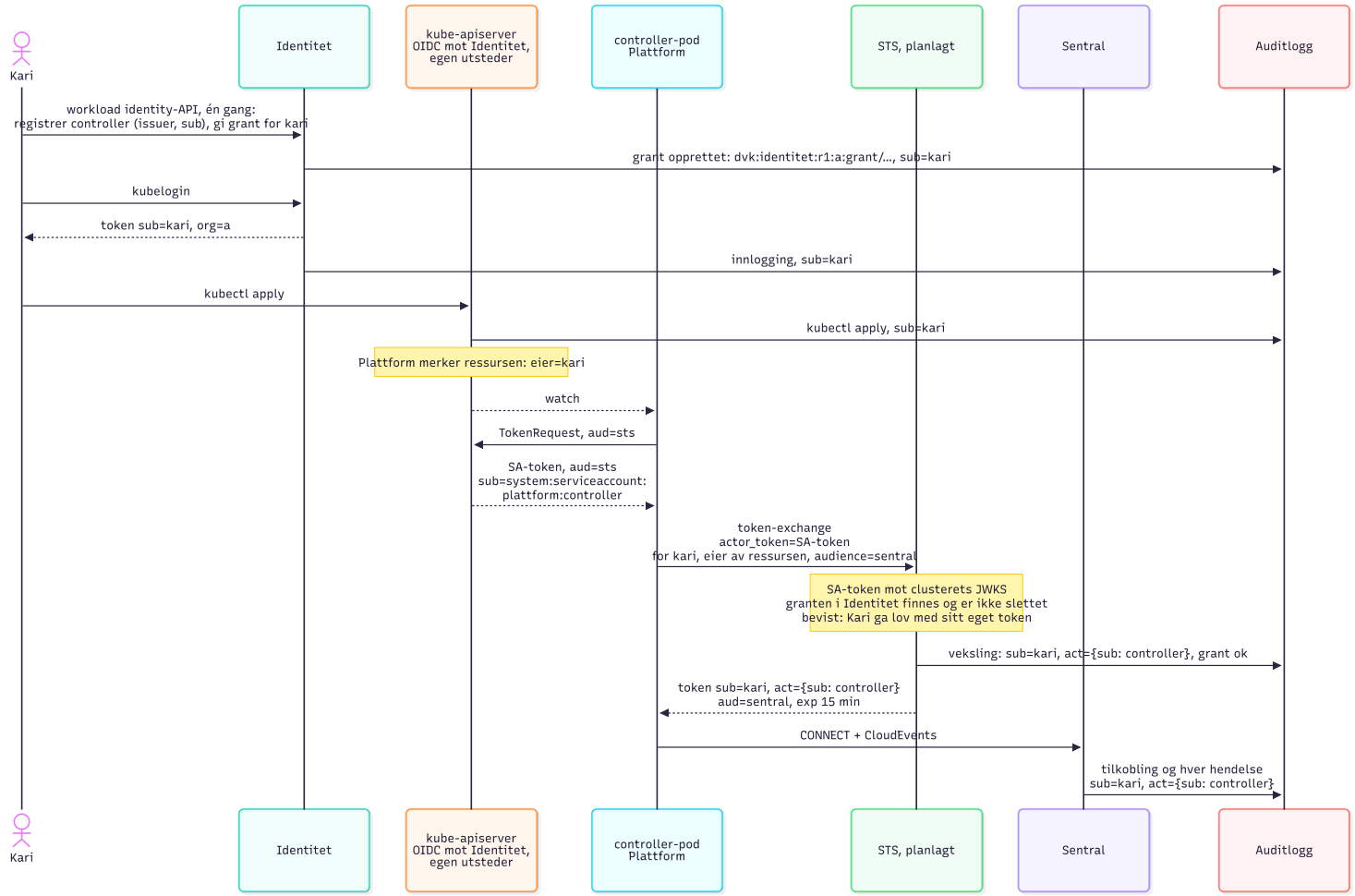
Sentral leser aldri `data`. Resten er nok til å rute, tillate og logge.

Konvolutten bestemmer, ikke innholdet

RBAC sjekker bare verb, ressurs og namespace. Sentral sjekker bare type, ressurs-ID og org.

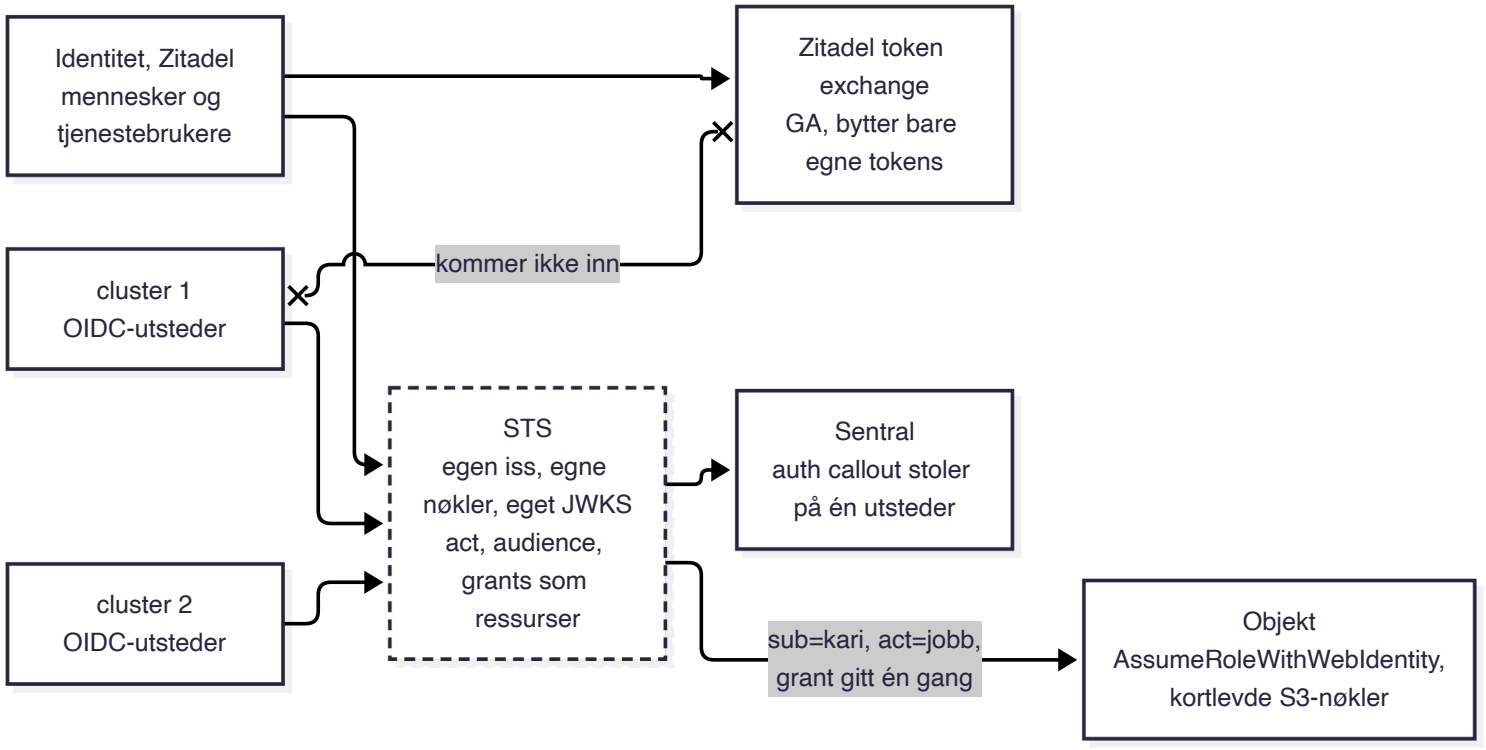


Løsning: workload-delegering med RFC 8693



Hullet

Den stiplede boksen finnes ikke i dag. Linus har begynt å skrive den. Samme token åpner en S3-bøtte hos Objekt, uten access key i en Secret.



Salmon

Vi er per dags dato blitt veldig gode på strømlinjeforming av menneskelig autentisering.
Men maskiner, not so much.

Eksempel fra min erfaring i NHN.

Alarmathan

Flere hundre instanser av Alertmanager, med integrasjoner mot Splunk, Zabbix og SCOM.

- mTLS, nei.
- API-nøkler, nei.
- Service accounts fra 2002, nei.
- OpenID, ja!

Hva salmon er

En identitet som ikke bare er menneskelig. En issuer for både mennesker og workloads.

- En IDP for identiteter, ikke bare mennesker
- Selvbetjening som hovedprinsipp
- Et ønske om å møte klienten på midten
- Sikkerhet og oversikt

Hvorfor orke et elegant oppsett nå?

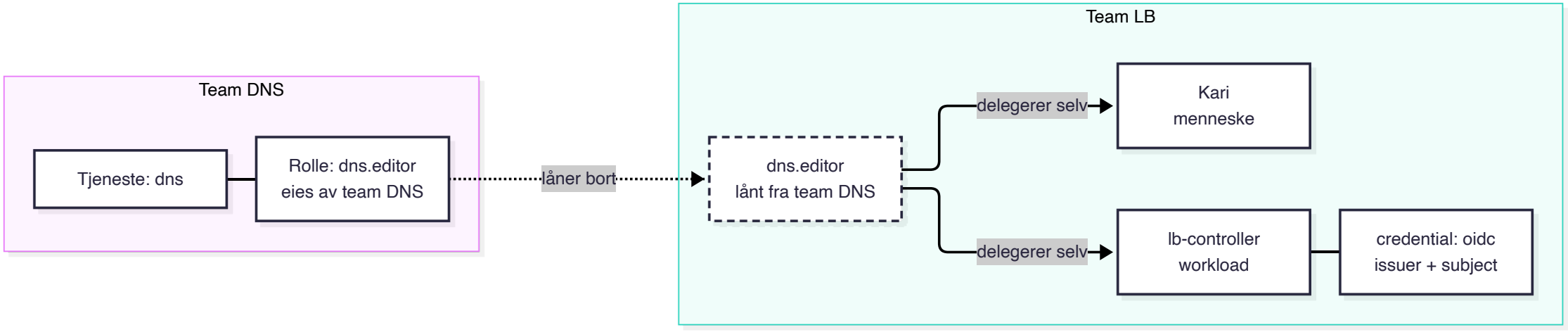
- Modellene skriver koden. Vi eier feilene
- En forutsigbar modell gir modellen mindre å gjette på, og dermed færre åpenbare feil
- Den enkleste veien bør også være den riktige: team, rolle, bevis, token
- Kortlevd identitet avgrenser en feil. En statisk nøkkel gjør det ikke

Datamodellen

Salmon er bygget rundt en team-drevet modell.

- Et team kan ha flere tjenester
- En tjeneste kan ha flere roller
- En rolle kan ha flere utlån
- Et team kan ha flere apps og identiteter
- En identitet kan ha flere roller

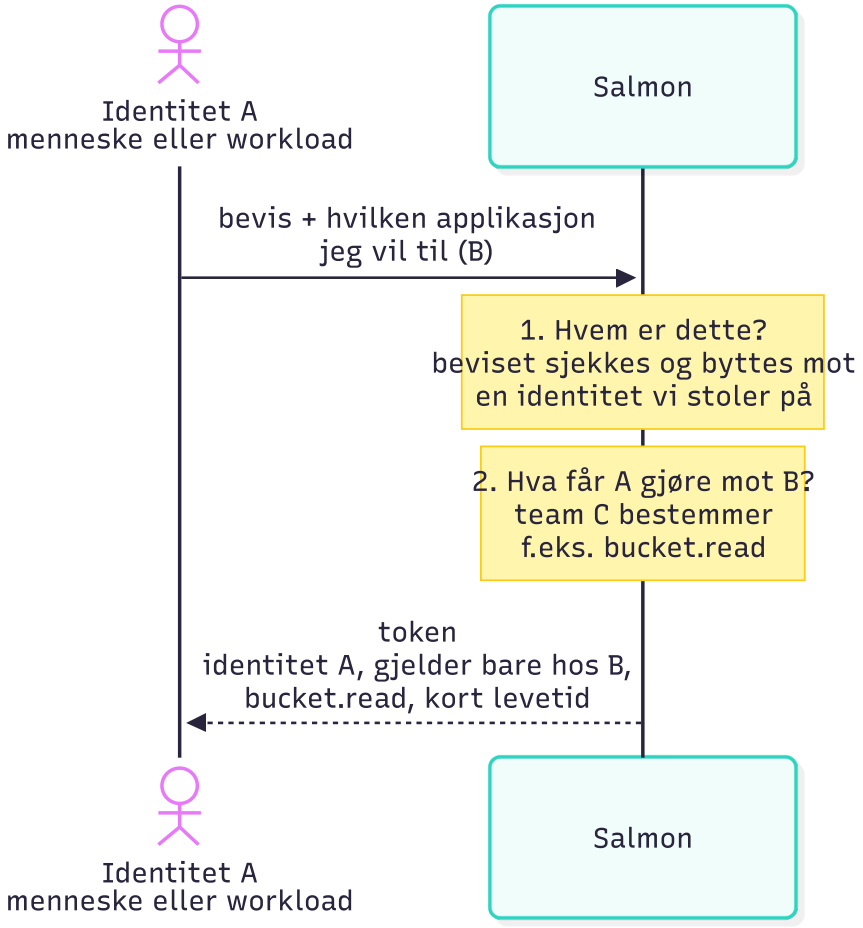
Visualisert



Token exchange, flyten

Kort forklart: Et bytte av et bevis mot en identitet noen andre stoler på.

Token exchange, grundig forklart



Hva er et bevis?

- En service account fra 2002?
- En SPIFFE-identitet (SVID)
- Et sertifikat
- En Kubernetes service account
- En potet, neida.

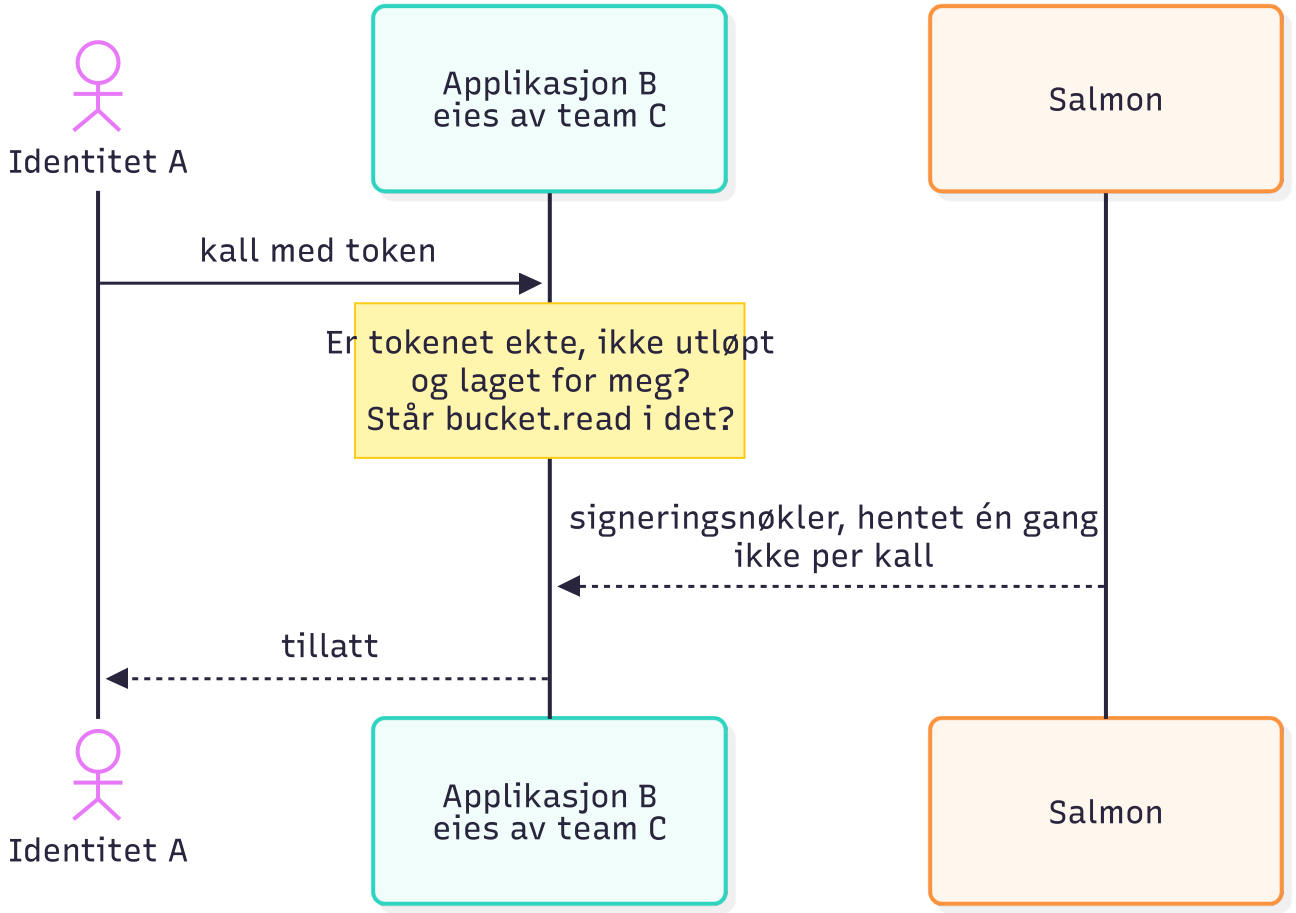
Under panseret: /token

```
curl -s https://zitadel.dataverket.org/oauth/v2/token \
  -d grant_type=urn:ietf:params:oauth:grant-type:token-exchange \
  -d subject_token="$BEVIS" \
  -d subject_token_type=urn:ietf:params:oauth:token-type:jwt \
  -d audience=applikasjon-b
```

```
{
  "access_token": "eyJhbGciOi…",
  "issued_token_type": "urn:ietf:params:oauth:token-type:access_token",
  "token_type": "Bearer",
  "expires_in": 900
}
```

Eksempel på formen, forkortet.

Applikasjonen verifiserer



B spør ikke Salmon for hvert kall. Svaret ligger i tokenet: hvem, hvilken applikasjon, hva.

Under panseret: hvorfor henter B openid-configuration?

```
curl -s https://zitadel.dataverket.org/.well-known/openid-configuration \
  | jq '{issuer, token_endpoint, jwks_uri}'
```

```
{
  "issuer": "https://zitadel.dataverket.org",
  "token_endpoint": "https://zitadel.dataverket.org/oauth/v2/token",
  "jwks_uri": "https://zitadel.dataverket.org/oauth/v2/keys"
}
```

1. Tokenet sier hvem som utstedte det (`iss`) og hvilken nøkkel som signerte det (`kid`)
2. B spør utstederen hvor nøklene er (`jwks_uri`)
3. B sjekker signatur, mottaker, og at tokenet ikke er utløpt

Visjonen: ett rammeverk for klienter

```
client, err := salmon.New().
    WithContext(ctx).
    WithAuth(salmon.OidcAuth(salmon.Kubernetes())).
    WithAudience("applikasjon-b")

app.
    WithContext(ctx).
    WithClient(client).
    WriteBucket("my-bucket", bucketData{})
```

Skisse, ikke implementert.

Hva som mangler

- Utlån av roller mellom team
- Roller og tilgang i tokenet, ikke bare hvem
- Mennesket bak maskinen: delegering som følger med
- Flere typer bevis: SPIFFE, sertifikater
- Rammeverk for klienter

Ønskelisten

Begge, 2 minutter

Til det norske åpen kildekode-fellesskapet

- Modne delegeringsprofiler i identitetsplattformene
- Standardisert identitet for meldinger
- Gjenbrukbare byggeklosser i et kontrollplan

Spørsmål og svar

Først oss, så dere?

Om Salmon, 1

Tokenet sier bare at workloaden finnes. Hva er det minste du vil ha med i det, og hva skal mottakeren avgjøre selv?

Om Salmon, 2

OpenBao stoler på clusteret i dag, og Keycloak 26.8 har act i preview. Hvorfor skrive en egen?

Om Salmon, 3

Tillit til en workload er én rad hos deg. Kan en grant være det samme, og hva må til for at et menneske kan være subjekt?

Om Dataverket, 1

Hvis Identitet får Keycloak ved siden av Zitadel, hvem er brukeren i Sentral?

Om Dataverket, 2

Hvorfor ikke bare OpenBao?

Om Dataverket, 3

Keycloak 26.8 kan sette act. Trenger dere fortsatt en egen STS?

Ett problem, to svar

Hvilken prosess på hvilken maskin er dette? En konto i Sentral som speiler organisasjonen, et token som utløper, en konvolutt som avgjør. Ingen hemmeligheter å rotere.

Hvem kjører hva på den? `sub` er mennesket som har ansvaret, `act` er prosessen som jobber på vegne av det, utstedt av en STS på en grant mennesket selv ga, slik workload identity gjør det hos skyleverandørene. STS-en finnes ikke ferdig i dag. Linus har begynt å skrive den.

Bli med i Dataverket: medlem@dataverket.org